LAB4 예비 보고서

STM32_0 LED·버튼 폴링 — FPGA 4비트 곱셈기·BCD→이진수 변환

과목 전자전기컴퓨터설계실험 2

제출일 2026년 10월 12일

성명 김태이

학번 2023440158

1 보고서 범위와 결론

10월 12일 실험(STM32_0)은 두 부분이다. NUCLEO-G474RE에서는 제공 예제 2개(1초 간격 LED, 버튼으로 LED 변경)를 실행한 뒤 과제1(버튼으로 점멸 간격을 1→2→4→8→1초로 변경)과 과제2(같은 동작을 외부 LED로 구현)를 직접 작성한다. Combo II-DLD에서는 교재 실습 4비트 곱셈기와 BCD→이진수 변환을 시뮬레이션하고 보드에서 시연한다[1].

이 보고서는 실험 전에 설계 목표와 방법, 핵심 코드와 회로 구조, 사전 검증 결과, 보드에서 확인할 항목을 정리한다. 과제1·2의 코드는 미리 작성해 Arm GCC로 빌드했고, 같은 소스 파일을 PC에서 HAL 대역과 함께 실행해 간격 순환, 길게 누름, 접점 떨림, tick 넘침의 네 경우를 확인했다. FPGA 두 설계는 Icarus Verilog와 Vivado 시뮬레이터에서 256개 입력 조합을 모두 통과했고, Vivado에서 bitstream 생성까지 마쳤다.

사전 검증에서 한 가지 위험을 찾았다. 제공 예제의 200 ms 잠금은 누르는 순간의 떨림만 막으므로, 200 ms보다 길게 눌렀다 뗄 때 떨림이 있으면 간격이 한 번 더 바뀐다. 실제 버튼에서 이 현상이 나오는지는 보드에서 확인할 항목이며, 나올 때 바꿔 넣을 대비안을 준비했다.

항목	사전 검증	결과
제공 예제 1·2	호스트 시뮬레이션	1초 간격, 누름당 1회
과제1	Arm GCC 빌드, 호스트 4개 시나리오	1→2→4→8→1초
과제2	Arm GCC 컴파일, 호스트	PB0에서 과제1과 같은 시각
곱셈기	Icarus·Vivado 256쌍	PASS, LUT 16개
BCD 변환	Icarus·Vivado 256개	PASS, LUT 7개

여기서 PASS와 호스트 시뮬레이션 결과는 논리가 맞다는 뜻이다. 핀 결선, LED·스위치의 극성, 실제 버튼의 떨림은 포함하지 않으므로 각 장 끝의 표대로 보드에서 따로 확인한다.

2 실험 환경과 검증 방법

구분	사용한 것
MCU 보드	NUCLEO-G474RE (STM32G474RE, Cortex-M4)
MCU 프로젝트	Glaysia/ece2_474 v1.1.0
MCU 빌드	Arm GNU Toolchain 13.3.Rel1, CMake 3.24.2, Ninja 1.11.1
호스트 검증	MinGW-W64 gcc 9.5.0, HAL 대역 헤더
FPGA 보드	Combo II-DLD, xc7s75fgga484-1
FPGA 도구	Vivado 2025.2, Icarus Verilog 12.0, fpga-lab-template v2.0.2

MCU 프로젝트는 수업에서 지정한 태그 v1.1.0을 받아 CMake의 Debug 구성으로 빌드했다[1]. 매뉴얼은 VS Code의 STM32CubeIDE 확장을 쓰지만, 이 PC에는 아직 확장이 없어 같은 CMake 프리셋을 명령줄에서 실행했다. 보드에 내려받는 과정(F5)은 실험실에서 수행한다.

보드가 없는 상태에서 동작을 확인하려고 호스트 시뮬레이션을 만들었다. 보드에 올릴 ece2_474_routine.c를 고치지 않고 PC용 gcc로 컴파일하되, stm32g4xx_hal.h 자리에 GPIO와 HAL_GetTick만 흉내 내는 대역 헤더를 넣는다. 메인 루프 한 바퀴를 50 µs로 두고 polling_routine()을 반복 호출하면서, 시나리오표의 시각에 버튼 입력을 바꾸고 출력 핀이 바뀐 시각을 기록한다. 실제 보드의 루프는 이보다 빠르지만 판단은 ms 단위 tick으로 하므로 결과 시각은 같다.

FPGA 실습은 매뉴얼에 실린 설계 모듈과 테스트벤치를 fpga-lab-template 폴더 구조에 넣어 Icarus Verilog로 실행하고, 같은 파일을 Vivado 2025.2 시뮬레이터에서 다시 실행해 파형 창을 캡처했다. 캡처는 한 단 폭에서 값이 읽히도록 시간축을 반으로 잘라 두 줄로 배치했다. 두 설계 모두 조합회로라 클록이 없다.

3 소스 출처와 실행 스냅샷

항목	식별 정보
MCU 템플릿	https://github.com/Glaysia/ece2_474.git
태그·커밋	v1.1.0 · dd94f01047aebdece09b9620c66e9b179403e5c9
FPGA 템플릿	Glaysia/fpga-lab-template v2.0.2
제출 저장소	https://github.com/tae2yi/UOS_ece2
LAB4 코드	570f69c767bab29c345ae9782cacd9e60b9184b3 (보고서의 GitHub 주소가 가리키는 위치)
폴더	lab4/stm32_lab, stm32_0_hostsim, lab4_01_mult_4bit, lab4_02_bcd_conv

GitHub:
https://github.com/tae2yi/UOS_ece2/tree/570f69c767bab29c345ae9782cacd9e60b9184b3/lab4

템플릿에서 바꾼 파일은 Core/Src/ece2_474_routine.c 하나다. 바뀐 내용 전체는 아래 diff에 있고, 호스트 시뮬레이션의 원자료(CSV)와 FPGA 시뮬레이션 로그도 같은 저장소에 있다. 시연 영상은 실험 후 보고서에 링크한다.

diff:
https://github.com/tae2yi/UOS_ece2/blob/570f69c767bab29c345ae9782cacd9e60b9184b3/lab4/report/pre/logs/stm32_routine.diff

STM32_0 폴링 구조와 제공 예제

설계 목표와 실행 흐름

목표는 버튼을 읽고 LED의 동작을 프로그램으로 바꾸는 것이다. 인터럽트는 쓰지 않는다. main.c의 while(1)이 polling_routine()을 쉬지 않고 호출하고, 함수는 호출될 때마다 현재 버튼 값과 현재 시각을 읽어 할 일이 있는지 판단한 뒤 바로 돌아온다. 기다리는 함수(HAL_Delay)를 쓰지 않으므로 LED 간격을 재는 동안에도 버튼을 놓치지 않는다.

시각은 HAL_GetTick()으로 읽는다. HAL은 SysTick 인터럽트에서 1 ms마다 tick 변수를 1씩 올리고 HAL_GetTick()은 그 값을 돌려준다[4]. SysTick은 Cortex-M 프로세서에 들어 있는 24비트 감산 타이머로, 이런 주기적 시간 기준을 만드는 용도로 쓴다[2]. 함수 안의 static 변수는 호출이 끝나도 값이 남으므로 이전 버튼 상태와 마지막 변경 시각을 여기에 둔다.

핀	이름	역할
PC13	BUTTON_ece2	사용자 버튼 입력. 누르면 HIGH
PA5	LED_ece2	내장 LED. HIGH에서 켜짐
PB0	GPIO_OUT1_ece2	범용 출력. 과제2의 외부 LED
PB1	GPIO_OUT2_ece2	범용 출력. PB0의 대안

PC13은 CubeMX 설정에서 내부 풀업·풀다운 없이 상승에지 인터럽트 입력으로 잡혀 있고, 제공 예제도 HIGH를 누름으로 본다. 폴링 실습에서는 인터럽트 함수(button_isr_p7)를 비워 두고 HAL_GPIO_ReadPin으로 값만 읽는다.

제공 예제 1·2의 동작

예제	판단 조건	동작
1	now − last ≥ 1000	led_state를 반전해 WritePin. 주기 2초
2	이전 LOW, 현재 HIGH, 직전 인정 후 200 ms 이상	TogglePin 한 번

예제 1은 시간이 조건이고 예제 2는 입력의 변화가 조건이다. 예제 2에서 현재 값만 보면 누르고 있는 동안 계속 토글되므로, 이전 값과 비교해 LOW에서 HIGH로 바뀐 한 순간만 고른다. 뺄셈 now − last는 부호 없는 32비트 연산이라 tick이 0xFFFFFFFF에서 0으로 넘어가도 경과 시간이 그대로 나온다.

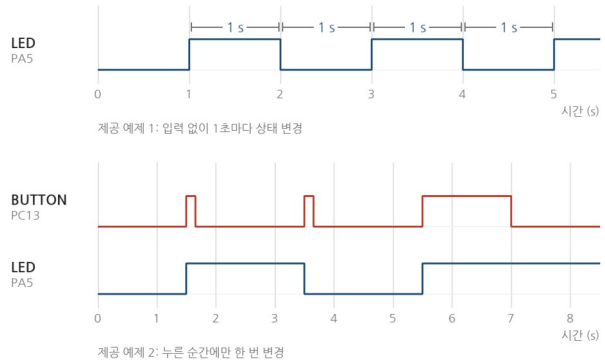


그림 1. 제공 예제 1·2의 호스트 시뮬레이션. 위는 1초마다 반전, 아래는 누른 순간에만 반전(1.5초 누름도 한 번).

STM32_0 과제1 버튼으로 LED 간격 변경

요구 조건과 설계

- 버튼을 누를 때마다 LED 상태 변경 간격이 1 → 2 → 4 → 8 → 1초로 바뀐다.
- 버튼을 누르지 않아도 선택한 간격으로 계속 깜빡인다.
- 버튼을 계속 누르고 있어도 간격은 한 번만 바뀐다.

매뉴얼의 안내대로 두 제공 예제를 합치고 간격을 저장하는 변수 interval_ms 하나를 더했다. 예제 2의 누름 검출이 LED를 토글하던 자리에서 interval_ms를 바꾸고, 예제 1의 고정값 1000이 있던 자리에 interval_ms를 넣는다. 두 판단은 서로 기다리지 않고 매 호출마다 차례로 실행된다.

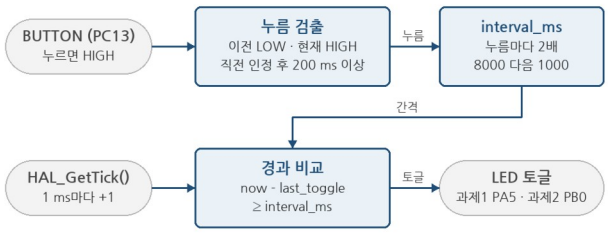


그림 2. 과제1의 처리 흐름. 버튼 쪽은 간격만 바꾸고, 시간 쪽은 그 간격으로 LED를 토글한다.

static 변수	초기값	역할
previous	RESET	직전 호출에서 읽은 버튼 값
last_press_ms	0	마지막으로 누름을 인정한 시각
last_toggle_ms	0	마지막으로 LED를 바꾼 시각
interval_ms	1000	현재 간격. 1000·2000·4000·8000

간격은 2배씩 늘다가 8000 이상이면 1000으로 돌아간다. 값 네 개를 배열에 두고 번호를 돌리는 방법도 있지만, 수열이 2의 거듭제곱이라 곱셈 한 번이 더 짧고 변수도 하나로 끝난다.

핵심 코드

```
void polling_routine(void)
{
  static GPIO_PinState previous = GPIO_PIN_RESET;
  static uint32_t last_press_ms = 0;
  static uint32_t last_toggle_ms = 0;
  static uint32_t interval_ms = 1000U;
  GPIO_PinState current =
    HAL_GPIO_ReadPin(BUTTON_ece2_GPIO_Port, BUTTON_ece2_Pin);
  uint32_t now_ms = HAL_GetTick();


  if (current == GPIO_PIN_SET && previous == GPIO_PIN_RESET
    && now_ms - last_press_ms >= 200U)
  {
    last_press_ms = now_ms;
    interval_ms = (interval_ms >= 8000U)
          ? 1000U : interval_ms * 2U;
  }
  previous = current;


  if (now_ms - last_toggle_ms >= interval_ms)
  {
    last_toggle_ms = now_ms;
    HAL_GPIO_TogglePin(BLINK_LED_GPIO_Port, BLINK_LED_Pin);
  }
}
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/570f69c767bab29c345ae9782cacd9e60b9184b3/lab4/stm32_lab/Core/Src/ece2_474_routine.c#L69-L93

LED는 WritePin 대신 TogglePin으로 바꾼다. 현재 상태를 따로 기억할 필요가 없고, 포트와 핀 이름을 BLINK_LED_* 한 곳에서 정하므로 과제2에서 이 함수는 손대지 않는다.

간격을 바꿔도 last_toggle_ms는 그대로 둔다. 따라서 바꾼 직후의 첫 변화는 누른 시각이 아니라 직전 LED 변화 시각에서 새 간격만큼 뒤에 온다. 8초에서 1초로 돌아갈 때 이미 1초가 지났다면 다음 호출에서 바로 바뀐다.

예상 결과: 간격 순환

버튼을 4.5, 10.5, 22.5, 46.5초에 150 ms씩 누르는 시나리오를 실행했다. LED가 바뀐 시각은 다음과 같다.

구간	LED가 바뀐 시각 (s)	간격
시작~4.5 s	1, 2, 3, 4	1 s
4.5~10.5 s	6, 8, 10	2 s
10.5~22.5 s	14, 18, 22	4 s
22.5~46.5 s	30, 38, 46	8 s
46.5 s 이후	47, 48, 49, 50, 51, 52	1 s

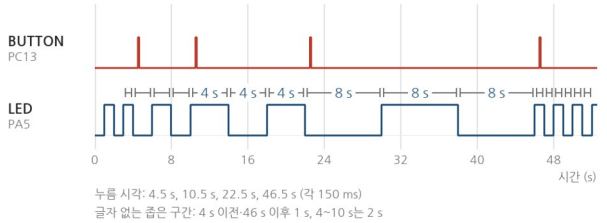


그림 3. 과제1 호스트 시뮬레이션. 누를 때마다 LED 변화 간격이 2, 4, 8초로 늘고 네 번째 누름에서 1초로 돌아간다.

원자료:
https://github.com/tae2yi/UOS_ece2/blob/570f69c767bab29c345ae9782cacd9e60b9184b3/lab4/stm32_0_hostsim/results/task1_cycle.csv

4.5초에 눌렀을 때 다음 변화가 5초가 아니라 6초에 온 것은 위에서 설명한 대로 직전 변화(4초)를 기준으로 2초를 세기 때문이다. 같은 입력을 tick이 0xFFFFB1E0에서 시작해 20초 뒤 0으로 넘어가도록 바꿔 실행해도 변화 간격은 같았다. 다만 이 경우 last_toggle_ms의 초기값 0과의 차이가 커서 시작하자마자 한 번 바뀐다.

예상 결과: 길게 누름

2.5초부터 6초 동안 누르고 있어도 간격은 2초로 한 번만 바뀌고, 누르고 있는 동안에도 LED는 2초마다 계속 바뀐다. 함수가 버튼을 기다리지 않기 때문이다.

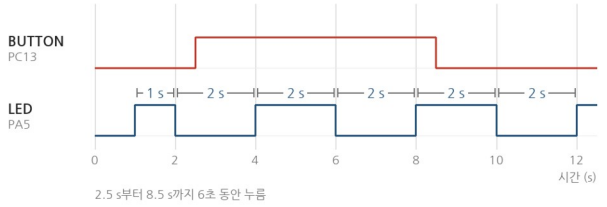


그림 4. 6초 동안 누른 경우. 간격은 1초에서 2초로 한 번만 바뀐다.

접점 떨림과 대비안

기계식 버튼은 누르거나 뗄 때 접점이 잠깐 붙었다 떨어지기를 반복할 수 있다. 에지마다 0.4 ms 간격으로 5번 떨리는 모델을 넣어 실행했다. 150 ms 누름에서는 누를 때의 떨림도, 150 ms 뒤 뗄 때의 떨림도 200 ms 잠금 안에 있어 한 번만 인정됐다. 1.5초 누름에서는 달랐다. 뗄 때는 잠금이 이미 풀려 있어서, 떨림 중 다시 HIGH가 되는 순간이 새 누름으로 인정됐다.

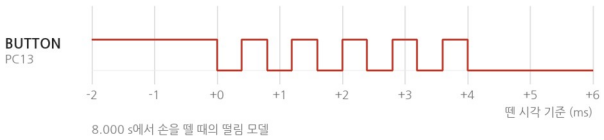


그림 5. 떼는 순간(8.000 s)의 떨림 모델. LOW가 된 뒤 다시 HIGH가 되는 상승 에지가 다섯 번 생긴다.

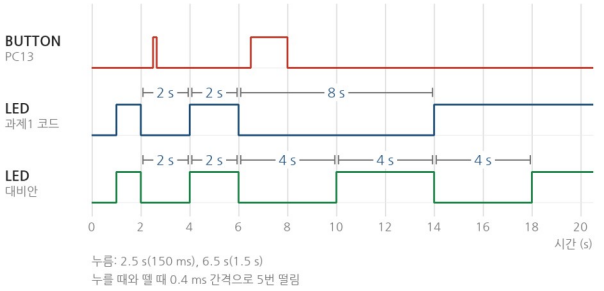


그림 6. 떨림이 있을 때. 과제1 코드는 두 번째 누름을 뗄 때 간격이 4초에서 8초로 한 번 더 바뀌고, 대비안은 4초를 유지한다.

대비안은 누름 인정 시각 대신 버튼 값이 마지막으로 바뀐 시각을 기억하고, 그로부터 50 ms 이상 조용했던 뒤의 상승만 누름으로 본다. 뗄 때의 하강도 시각을 갱신하므로 바로 뒤따르는 떨림은 걸러진다.

```
if (current != previous)
{
  if (current == GPIO_PIN_SET
    && now_ms - last_change_ms >= 50U)
  {
    interval_ms = (interval_ms >= 8000U)
          ? 1000U : interval_ms * 2U;
  }
  last_change_ms = now_ms;
  previous = current;
}
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/570f69c767bab29c345ae9782cacd9e60b9184b3/lab4/stm32_0_hostsim/variants/task1_release_guard.c#L20-L31

이 떨림은 모델이지 측정이 아니다. NUCLEO 보드의 버튼 회로에 떨림을 줄이는 소자가 있으면 과제1 코드만으로 충분할 수 있다. 그래서 매뉴얼의 구성에 가까운 과제1 코드를 먼저 올리고, 길게 눌렀다 뗄 때 간격이 두 단계 넘어가는 경우에만 대비안으로 바꾼다. 대비안은 떨림이 없는 순환·길게 누름 시나리오에서 과제1 코드와 같은 결과를 냈다.

빌드 결과

```
$ cmake --preset Debug
$ cmake --build --preset Debug
RAM:    2656 B / 128 KB  (2.03 %)
FLASH: 25848 B / 512 KB  (4.93 %)
polling_routine  0x080056d4  152 B
```

로그:
https://github.com/tae2yi/UOS_ece2/blob/570f69c767bab29c345ae9782cacd9e60b9184b3/lab4/report/pre/logs/stm32_arm_build.txt

Arm GNU Toolchain 13.3으로 빌드했고 ece2_474_routine.c에서 경고는 없었다. interval_ms는 0이 아닌 초기값이 있어 .data에, 나머지 세 변수는 .bss에 놓였다.

보드에서 확인할 항목

확인 항목	예상 관찰
제공 예제 1	내장 LED가 1초 켜지고 1초 꺼짐
제공 예제 2	누를 때마다 LED 상태가 한 번 바뀜
과제1 시작	리셋 후 1초 간격
누름 1~3회	2초, 4초, 8초 간격. 스톱워치로 확인
누름 4회	다시 1초 간격
길게 누름	간격이 한 단계만 바뀜. 두 단계면 대비안 적용
누르는 동안	LED가 멈추지 않고 계속 깜빡임
다운로드	Build만으로는 바뀌지 않음. 디버깅 종료 후 F5

STM32_0 과제2 외부 LED로 동작시키기

회로 구성

과제1의 동작을 PB0에 연결한 외부 LED로 옮긴다. PB0는 push-pull 출력으로 설정되어 있고 초기값은 LOW다. 핀에서 저항과 LED를 거쳐 GND로 가는 직렬 회로이므로 PB0가 HIGH일 때 켜진다. 내장 LED와 극성이 같아 코드의 논리를 뒤집을 필요가 없다.



그림 7. 과제2 결선. PB0 → 저항 → LED 긴 다리, LED 짧은 다리 → GND. 커넥터 위치는 보드 설명서로 다시 확인한다.

저항	계산	LED 전류
330 Ω	(3.3 − 2.0) / 330	약 3.9 mA
1 kΩ	(3.3 − 2.0) / 1000	약 1.3 mA

적색 LED의 순방향 전압을 약 2.0 V로 가정한 값이다. 저항이 클수록 어둡지만 두 값 모두 켜진 것을 알아보기에 충분하다. 저항 없이 연결하면 전류를 제한하는 것이 없으므로 반드시 저항을 넣는다.

코드 변경

polling_routine()은 그대로 두고, 파일 위쪽에서 LED 핀을 고르는 매크로 값만 0에서 1로 바꾼다.

```
#ifndef STM32_0_USE_EXTERNAL_LED
#define STM32_0_USE_EXTERNAL_LED 0
#endif


#if STM32_0_USE_EXTERNAL_LED
#define BLINK_LED_GPIO_Port GPIO_OUT1_ece2_GPIO_Port
#define BLINK_LED_Pin GPIO_OUT1_ece2_Pin
#else
#define BLINK_LED_GPIO_Port LED_ece2_GPIO_Port
#define BLINK_LED_Pin LED_ece2_Pin
#endif
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/570f69c767bab29c345ae9782cacd9e60b9184b3/lab4/stm32_lab/Core/Src/ece2_474_routine.c#L54-L64

main.h에서 PB0의 이름은 GPIO_OUT1_ece2_Pin과 GPIO_OUT1_ece2_GPIO_Port다. PB1을 쓰려면 GPIO_OUT2_ece2로 바꾼다. 값을 1로 두고 Arm GCC로 컴파일한 결과에서 HAL_GPIO_TogglePin의 핀 인자가 0x20(5번 핀)에서 0x01(0번 핀)로 바뀐 것을 확인했다.

예상 결과

과제1과 같은 시나리오에서 PB0가 바뀐 시각은 과제1의 PA5와 모두 같았고, PA5는 LOW로 남았다.

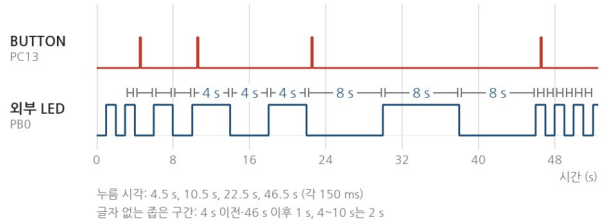


그림 8. 과제2 호스트 시뮬레이션. 외부 LED 핀 PB0가 과제1과 같은 시각에 바뀐다.

보드에서 확인할 항목

확인 항목	예상 관찰
결선	PB0 → 저항 → LED(+), LED(−) → GND
과제1 먼저	내장 LED로 과제1을 확인한 뒤 진행
외부 LED	리셋 후 1초 간격으로 깜빡임
버튼	외부 LED도 1→2→4→8→1초로 바뀜
내장 LED	꺼진 채로 유지
안 켜질 때	LED 방향, GND 연결, 매크로 값, F5 다운로드 순으로 확인

FPGA-01 4비트 곱셈기

회로 목적과 구조

부호 없는 4비트 수 a와 b를 곱해 8비트 m을 낸다. 가장 큰 결과가 15 × 15 = 225이므로 8비트(최대 255)면 넘치지 않는다. 입력만으로 출력이 정해지는 조합회로이며 클록과 리셋이 없다.

이진 곱셈은 손으로 하는 곱셈과 같다. 승수의 각 비트와 피승수를 AND해 부분곱을 만들고, 한 자리씩 옮겨 더한다. 4비트끼리의 곱은 AND 게이트 16개와 4비트 가산기 3개로 구성할 수 있다[3]. 설계에서는 이를 직접 그리지 않고 a * b로 적어 합성 도구가 회로를 만들게 했다.

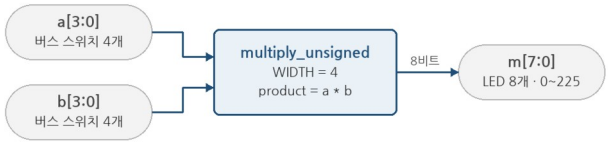


그림 9. FPGA-01 블록 흐름. 설계 top은 mult_4bit.

```
module mult_4bit(input wire [3:0] a, b, output wire [7:0] m);
  multiply_unsigned #(.WIDTH(4)) u_multiply(a, b, m);
endmodule


module multiply_unsigned #(parameter WIDTH = 4)(
  input wire [WIDTH-1:0] a, b,
  output wire [2*WIDTH-1:0] product
);
  assign product = a * b;
endmodule
```

GitHub:
https://github.com/tae2yi/UOS_ece2/tree/570f69c767bab29c345ae9782cacd9e60b9184b3/lab4/lab4_01_mult_4bit/src

곱셈은 multiply_unsigned가 하고 mult_4bit은 폭을 4로 정해 연결만 한다. 출력 폭을 2 × WIDTH로 선언했기 때문에 곱의 상위 비트가 잘리지 않는다.

파일의 역할

파일	역할
src/multiply_unsigned.v	폭을 매개변수로 받는 곱셈
src/mult_4bit.v	설계 top. 4비트 입출력 연결
sim/tb.sv	매뉴얼의 TB. 256쌍 전수 검사
sim/tb_view.sv	파형 확인용. 경계값 12쌍을 10 ns씩
constraints/mult_4bit.xdc	a·b 스위치 8핀, m LED 8핀, LVCMOS33

tb.sv는 매뉴얼의 코드에 파형 기록($dumpfile, $dumpvars) 한 줄만 더했다. 전수 검사는 1 ns마다 입력이 바뀌어 값을 읽기 어려우므로, 값을 눈으로 확인할 tb_view.sv를 따로 작성했다.

시뮬레이션 결과

```
[Icarus] PASS multiplier: 256 input pairs
    $finish called at 256000 (1ps)
[Vivado] PASS multiplier: 256 input pairs
    $finish called at time : 256 ns
[Vivado] PASS multiplier view: 12 directed pairs
```

로그:
https://github.com/tae2yi/UOS_ece2/tree/570f69c767bab29c345ae9782cacd9e60b9184b3/lab4/report/pre/logs

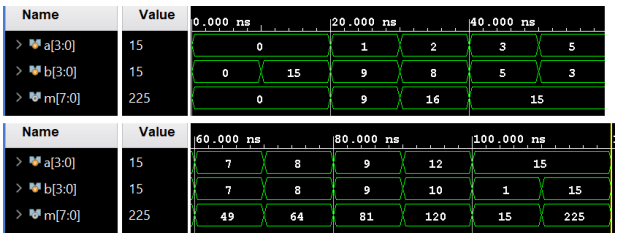


그림 10. FPGA-01 Vivado 파형(tb_view, 10진 표시). 캡처를 0~60 ns와 60~120 ns 두 줄로 나눠 놓았다. 0×15=0, 3×5=5×3=15, 12×10=120, 15×15=225.

두 시뮬레이터 모두 256쌍을 통과했다. 파형에서 한쪽 입력이 0이면 결과가 0이고, 3×5와 5×3이 같으며, 최댓값 225에서 m[7:0]이 1110_0001이 되는 것을 확인했다.

구현 결과

항목	값
LUT	16개 (DSP 0, 레지스터 0)
가장 긴 경로	a[0] → m[7], 12.063 ns
DRC	오류 0
bitstream	bitstream/mult_4bit.bit

가장 긴 경로는 최하위 입력에서 최상위 출력까지이며, 부분곱의 자리올림이 끝까지 전달되는 길과 일치한다. 이 지연은 입출력 버퍼를 포함한 값이고, 스위치를 손으로 바꾸는 실험에서는 의미 있는 제약이 아니다.

보드에서 확인할 항목

a	b	m (10진)	LED m[7:0]
0000	1111	0	0000_0000
0011	0101	15	0000_1111
1000	1000	64	0100_0000
1100	1010	120	0111_1000
1111	1111	225	1110_0001

XDC의 핀은 LAB1의 4비트 가산기(스위치)와 LAB3의 LED 배치를 그대로 썼다. 교재의 곱셈기 핀 표와 다르면 교재를 따른다. 스위치와 LED의 극성이 반대로 보이면 보드 회로를 확인한다.

FPGA-02 BCD→이진수 변환

회로 목적과 구조

BCD는 10진수의 각 자리를 4비트로 따로 적는 코드다[3]. 십의 자리 ten과 일의 자리 one을 받아 그 값 ten × 10 + one을 7비트 이진수 bin으로 낸다. 유효한 입력의 최댓값이 99(110_0011)이므로 7비트면 충분하다.

4비트 입력은 10~15도 나타낼 수 있지만 BCD에서는 쓰지 않는 값이다. 매뉴얼의 설계는 두 자리가 모두 0~9일 때만 valid를 1로 내도록 원본 예제에 출력을 하나 더 했다[1].

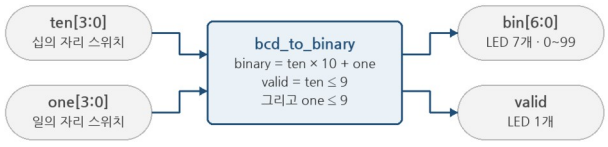


그림 11. FPGA-02 블록 흐름. 설계 top은 bcd_conv.

```
module bcd_to_binary(
  input wire [3:0] ten, one,
  output wire [6:0] binary,
  output wire valid
);
  assign binary = ten * 7'd10 + one;
  assign valid = (ten <= 9) && (one <= 9);
endmodule
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/570f69c767bab29c345ae9782cacd9e60b9184b3/lab4/lab4_02_bcd_conv/src/bcd_to_binary.v

곱하는 수를 7'd10으로 적어 곱셈이 7비트 폭에서 계산되게 했다. ten × 10은 ten × 8 + ten × 2, 곧 자리 옮김 두 번과 덧셈으로 만들 수 있어 실제 곱셈기가 필요하지 않다. top인 bcd_conv는 포트 순서(one, ten)만 바꿔 연결한다.

범위를 벗어난 입력

ten	one	계산값	bin	valid
9	9	99	99	1
10	0	100	100	0
0	10	10	10	0
12	7	127	127	0
15	15	165	37	0

valid가 0일 때도 bin에는 계산값이 그대로 나오고, 127을 넘으면 하위 7비트만 남는다(165 − 128 = 37). 따라서 bin은 valid가 1일 때만 믿을 수 있다. 일의 자리에 10을 넣으면 bin이 10이 되어 유효한 입력 "1, 0"과 구별되지 않는데, 이를 가르는 것이 valid다.

파일의 역할

파일	역할
src/bcd_to_binary.v	변환과 유효성 검사
src/bcd_conv.v	설계 top. 입출력 연결
sim/tb.sv	매뉴얼의 TB. 유효 100개, 무효 156개
sim/tb_view.sv	파형 확인용. 유효 8개, 무효 4개
constraints/bcd_conv.xdc	ten·one 스위치 8핀, bin·valid LED 8핀

시뮬레이션 결과

```
[Icarus] PASS BCD: 100 valid and 156 invalid inputs
    $finish called at 256000 (1ps)
[Vivado] PASS BCD: 100 valid and 156 invalid inputs
    $finish called at time : 256 ns
[Vivado] PASS BCD view: 12 directed inputs
```

로그:
https://github.com/tae2yi/UOS_ece2/tree/570f69c767bab29c345ae9782cacd9e60b9184b3/lab4/report/pre/logs

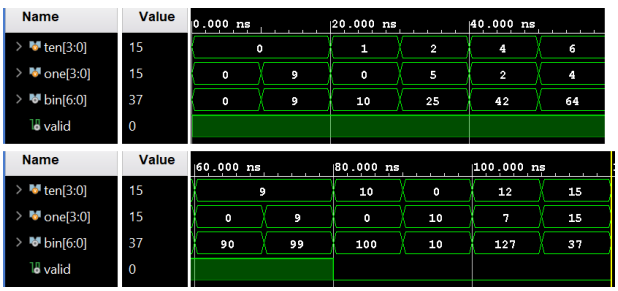


그림 12. FPGA-02 Vivado 파형(tb_view, 10진 표시, 두 줄 배치). 80 ns까지는 유효 입력으로 valid=1, 이후 네 입력은 valid=0.

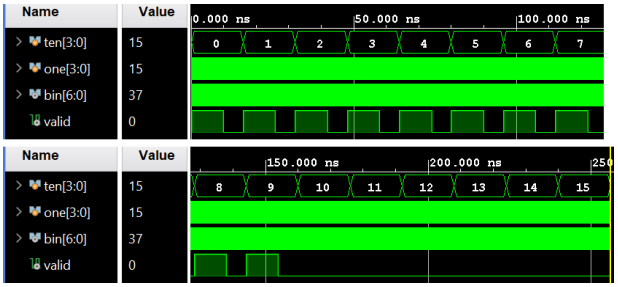


그림 13. FPGA-02 Vivado 전체 파형(tb, 256 ns, 두 줄 배치). ten이 0~9인 동안 one이 0~9인 구간에서만 valid가 1이고, ten이 10 이상이면 계속 0이다. one과 bin은 1 ns마다 바뀌어 띠로 보인다.

매뉴얼의 TB는 valid를 256개 입력 모두에서, bin을 유효한 100개에서 검사한다. 무효 입력의 bin은 검사하지 않으므로 tb_view.sv에서 계산값의 하위 7비트와 같은지 따로 확인했다.

구현 결과

항목	값
LUT	7개 (DSP 0, 레지스터 0)
가장 긴 경로	ten[0] → bin[6], 11.226 ns
DRC	오류 0
bitstream	bitstream/bcd_conv.bit

보드에서 확인할 항목

ten	one	bin[6:0]	valid
0000	0000	000_0000 (0)	켜짐
0100	0010	010_1010 (42)	켜짐
1001	1001	110_0011 (99)	켜짐
0100	1010	011_0010 (50)	꺼짐
1010	0000	110_0100 (100)	꺼짐

valid는 LED 8개 중 가장 왼쪽(곱셈기의 m[7] 자리)에 두었다. 교재의 핀 표와 다르면 교재를 따른다.

4 예상 결과 정리와 실험 계획

MCU와 FPGA 실습은 같은 일을 다른 방식으로 한다. 곱셈기와 BCD 변환기는 입력이 바뀌면 회로가 곧바로 출력을 내고, STM32에서는 프로그램이 입력과 시각을 반복해서 읽어 출력을 바꾼다. 과제1에서 버튼 처리와 LED 처리가 서로 막지 않는 것은 함수가 기다리지 않고 매번 돌아오기 때문이며, 이 구조가 다음 주의 인터럽트 방식과 비교할 기준이 된다.

순서	할 일	기록할 것
1	제공 예제 1·2 실행	LED 동작 영상
2	과제1 다운로드	네 간격을 스톱워치로 측정
3	길게 누름 시험	간격이 몇 단계 바뀌는지
4	과제2 결선·다운로드	외부 LED 영상, 결선 사진
5	곱셈기 bitstream	표의 다섯 입력과 LED
6	BCD 변환 bitstream	표의 다섯 입력과 LED

실험 후 보고서에서는 위 측정값을 이 보고서의 예상 시각·예상 LED 값과 비교하고, 길게 누름에서 떨림 문제가 실제로 나왔는지와 그때의 해결 과정을 적는다. 제출 버전의 태그와 커밋 해시, 시연 영상 링크도 그때 함께 적는다.

참고 문헌

[1] 이해리, 「STM32 실습 매뉴얼」(STM32_0, FPGA 추가 실습) 및 「전자전기컴퓨터설계실험 II STM32 실습 안내」, 서울시립대학교, 2026. https://github.com/Glaysia/ece2_474 (v1.1.0).

[2] Joseph Yiu, The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors, 3rd ed., Newnes, 2013. ISBN 978-0-12-408082-9.

[3] M. Morris Mano and Michael D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog, 6th ed., Pearson, 2018. ISBN 978-0-13-454989-7.

[4] STMicroelectronics, STM32CubeG4 MCU Firmware Package (stm32g4xx_hal.c, stm32g4xx_hal_gpio.c). https://github.com/STMicroelectronics/STM32CubeG4